Sysadmin Simulation

File Server Incident Report — A Learning Exercise | 15/06/2026

1. Introduction

This document is a write-up of a simulated sysadmin incident scenario, run as a learning exercise with AI assistance. The purpose is to practise real-world incident response skills in a safe environment — working through the problem step by step, using the right commands, and thinking through the implications of each action before taking it.

The scenario was presented as a live situation. I was not given the answers in advance — I had to diagnose and resolve the issue using my own knowledge, and where I was unsure, I consulted man pages or reference material before proceeding. This document reflects that process honestly, including the moments where I had to look things up.

2. The Scenario

Called into a small company at 08:30. Staff cannot access shared files on the Linux file server. The server is powered on and responsive. A junior member of staff had been working on the server the previous evening — he is not yet in. The office manager is stressed and the MD is asking questions.

3. Investigation

Step 1 — Establish who I am logged in as

First step was to confirm my user context before doing anything else:

```
whoami
```

Output: *sysadmin* — a standard user with sudo access. Not root directly.

Step 2 — Check running processes

Checked what was running on the system to look for anything unusual:

```
ps aux
```

Immediately spotted a process running as root with 99% CPU usage, started at 00:23 — nearly 8 hours earlier. The command was:

```
find / -name "*.log" -delete
```

A find command running as root, deleting every log file on the entire system, thrown into the background. Still running.

Step 3 — Confirm via command history

Checked the shell history to confirm what had been run:

```
history
```

Line 5 confirmed it: *sudo find / -name "*.log" -delete &* — the & at the end sent it to the background, which is why it kept running after the junior logged off.

4. Root Cause

A junior member of staff with unrestricted sudo access ran a recursive find command intended to free up disk space. The command deleted every file matching *.log across the entire filesystem, including Samba's active log files. Samba lost its log file around 02:17 and failed to recover, causing all shared file access to drop. The process continued running in the background for nearly 8 hours before being identified.

5. Resolution

Step 1 — Kill the rogue process

The process ID was 1042 from the ps aux output. A standard kill was tried first:

```
kill 1042
```

The process ignored it. Used SIGKILL to force terminate:

```
kill -9 1042
```

Process terminated. CPU usage returned to normal immediately.

Step 2 — Assess the damage

Checked the system journal for errors:

```
journalctl -xe
```

Confirmed Samba had been logging errors since 02:17 — unable to open its log file. journalctl itself was unaffected as it logs to a binary journal, not flat log files.

Step 3 — Restart Samba

Restarted the Samba service to allow it to recreate its log files:

```
sudo systemctl restart smbd
sudo systemctl status smbd
```

Samba came back up immediately. Staff confirmed shared file access was restored.

Step 4 — Restore deleted logs from backup

Checked for available backups:

```
ls /var/backups/
```

A nightly backup had run at 23:00 — one hour before the junior ran his command. Restored from the most recent backup:

```
sudo tar -xzf /var/backups/logs-backup-20260614.tar.gz -C /var/log/
```

Logs restored to their state as of 23:00 the previous evening. The one hour window between the backup and the incident was unrecoverable, but no company data was affected — only log files.

6. Recommendations to Management

- **Review sudo access immediately.** The junior had unrestricted sudo access — enough to run a destructive command across the entire filesystem as root. sudo permissions should be locked down to only what each user needs for their role.
- **Principle of least privilege.** No user should have more access than necessary to do their job. A junior doing routine disk cleanup should not be able to affect system-wide files.
- **Change the junior's password.** Pending a full access review, his credentials should be changed immediately.
- **Review the backup schedule.** A one-hour gap between the nightly backup and the incident was fortunate. Consider more frequent snapshots for critical systems.
- **Document what happened.** This incident should be logged formally so there is a record if it recurs or if questions are asked later.

7. What I Learned

The diagnostic process came fairly naturally — whoami, ps aux, and history felt like logical first steps. Spotting the rogue process in ps aux output and connecting it to the history entry was satisfying when it clicked.

Using kill -9 was the right call once the standard kill was ignored, but it's worth noting that SIGKILL should always be a last resort — it gives the process no chance to clean up. In this case the process was already causing damage so there was no reason to be gentle.

The tar restore command was where I had to stop and look things up. I knew tar was the right tool but I wasn't confident on the exact flags needed for this situation. I used *man tar* first to understand the available options, and cross-referenced with a Google search to confirm the correct combination for extracting a gzipped archive to a specific directory. In a real production environment this is exactly the right approach — running the wrong flags on a restore operation could make things worse. Looking it up before running it isn't a weakness, it's how you avoid turning a bad situation into a worse one.

I'll be honest — I couldn't repeat the exact tar syntax from memory right now without looking it up again. I understand what each flag does and why they're needed in this context, but that understanding needs reinforcing through repeated practice before it becomes instinct. More simulations like this one are the plan.

The recommendations section also reinforced something I've been reading about: the principle of least privilege isn't just a security concept — it's what prevents a well intentioned junior from accidentally taking down a file server at midnight. Access control is a practical operational concern, not just a compliance checkbox.

Overall this simulation was more valuable than I expected. Working through a problem under a degree of pressure — even a simulated one — is different from reading about the commands in a course. The decisions felt real enough to be useful practice.

This document was produced as part of an ongoing self-directed learning programme in Linux systems administration and DevOps/DevSecOps. All scenarios are simulated.